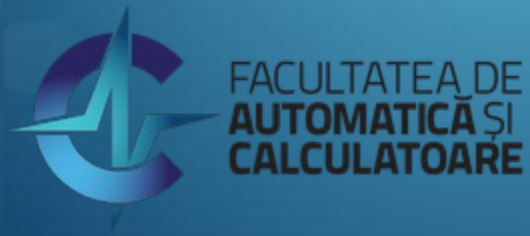


Algoritmi Paraleli și Distribuiți
Replicare, partiționare, consistență, consens.

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro



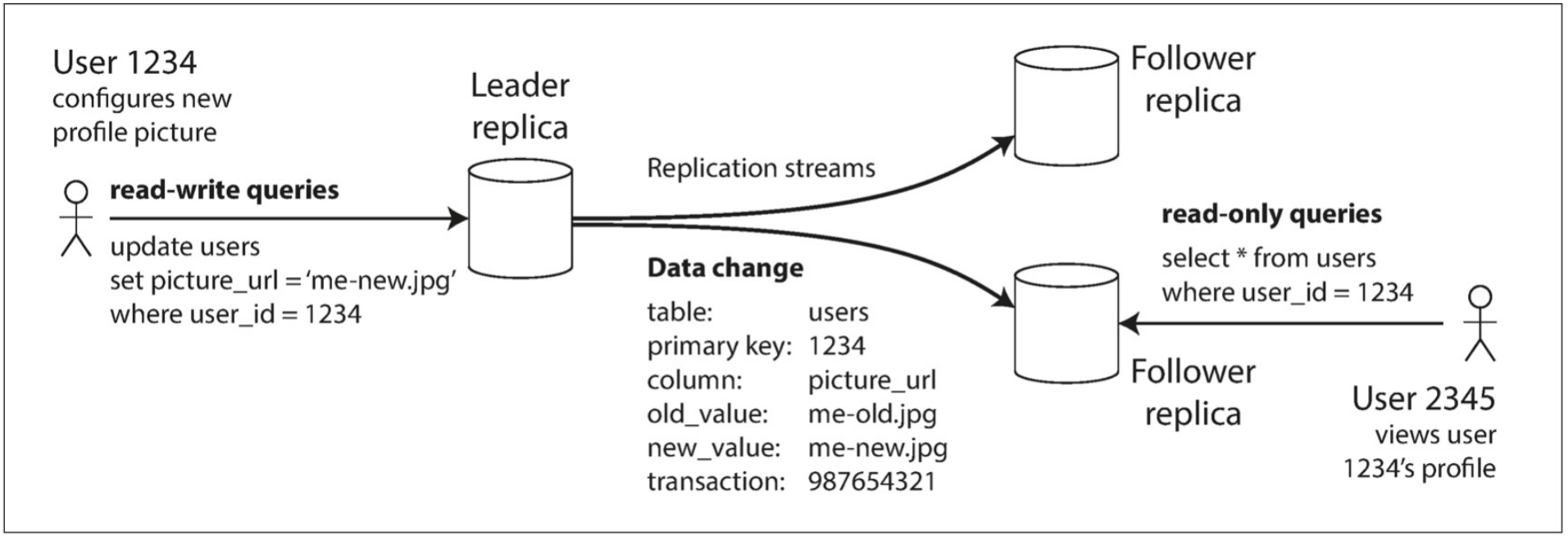
Replicare

- **Replicare**
  - stocarea mai multor copii ale acelorași date pe mai multe noduri conectate printr-o rețea
  - motivație:
    - ținerea datelor mai aproape de utilizatori (geografic) → reducerea latenței, funcționarea și atunci când au loc deconectări
    - funcționarea sistemului chiar dacă părți din el nu mai funcționează → creșterea disponibilității
    - scalarea numărului de noduri care pot răspunde la cereri → creșterea throughput-ului la citire, scalabilitate

Replicare (2)

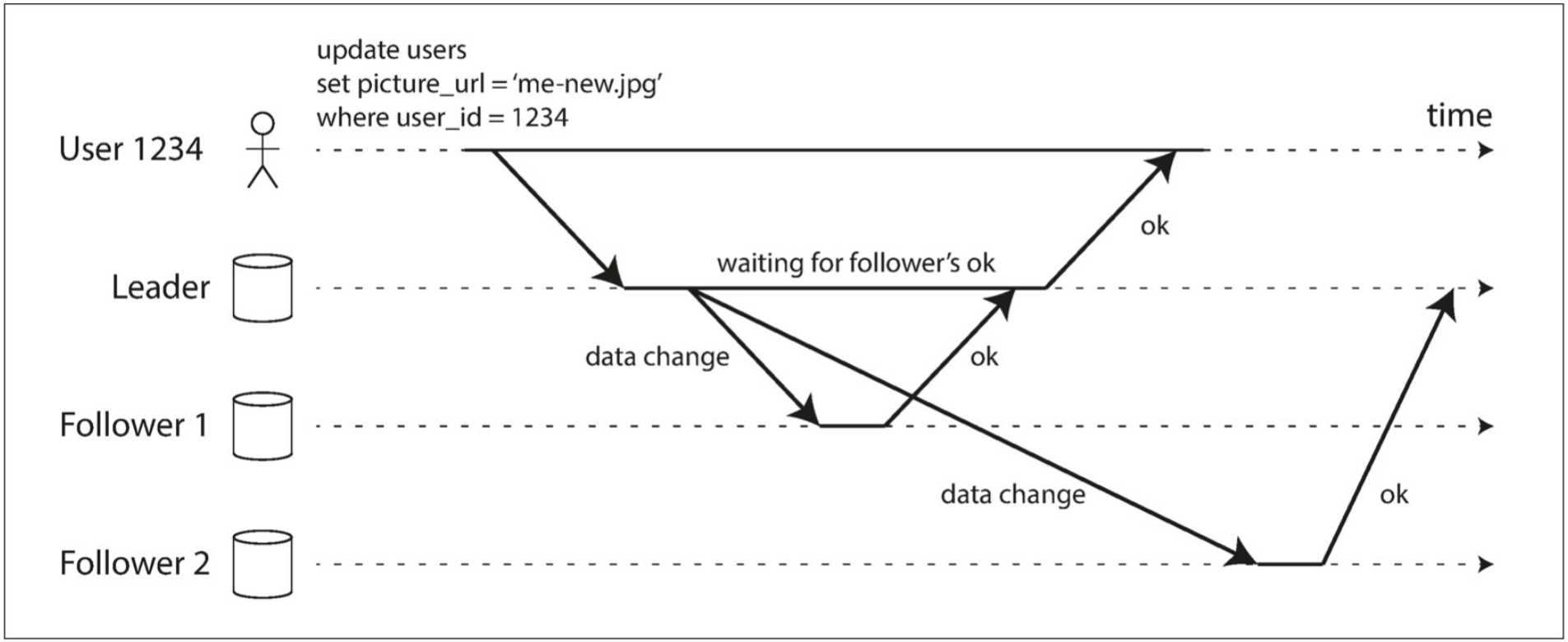
- **Replică** → un nod care conține o copie a unor date (a unei baze de date)
- Toate replicile trebuie să conțină toate datele → **cum ne putem asigura că toate datele sunt scrise corect pe toate replicile?**
  - replicare cu un lider
  - replicare cu mai mulți lideri
  - replicare fără lider

Replicare cu un lider



- Una din replici este **lider** → toate cererile de scriere se fac către această replică
- Toate celelalte replici sunt **urmăritori** → liderul informează toți urmăritorii despre fiecare scriere, urmăritorii își actualizează copia locală a datelor
- Un client care vrea să citească din baza de date poate trimite cererea către orice replică

Replicare sincronă și asincronă



- Replicare **sincronă** → liderul așteaptă confirmare de la urmăritor că scrierea a fost făcută înainte să-i confirme utilizatorului scrierea
- Replicare **asincronă** → liderul trimite mesajul către urmăritor, dar nu așteaptă confirmare

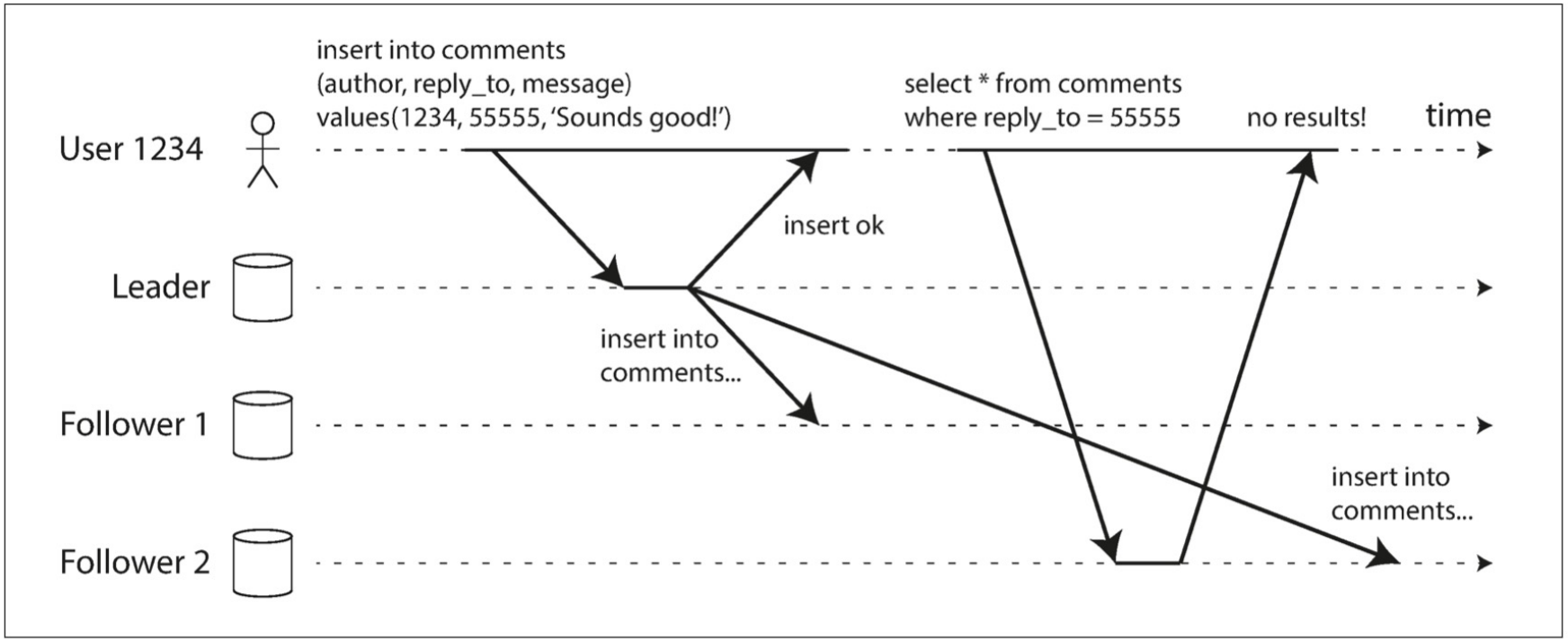
Tratarea eșecurilor nodurilor

- Eșecul unui urmăritor
  - fiecare urmăritor stochează un log cu modificările primite de la lider
  - când un urmăritor este inactiv/inaccesibil pentru o perioadă de timp, la repornire îi cere liderului toate modificările care au avut loc cât timp a fost inactiv, și apoi le aplică
- Eșecul liderului → **failover**
  - unul din urmăritori trebuie promovat ca noul lider
  - clienții trebuie să știe cine este noul lider (pentru a scrie la acesta)
  - ceilalți urmăritori trebuie să știe cine este noul lider (pentru a consuma modificări de date de la acesta)
  - **Ce se poate întâmpla la failover dacă folosim replicare asincronă?**

Probleme cu întârzierea replicării

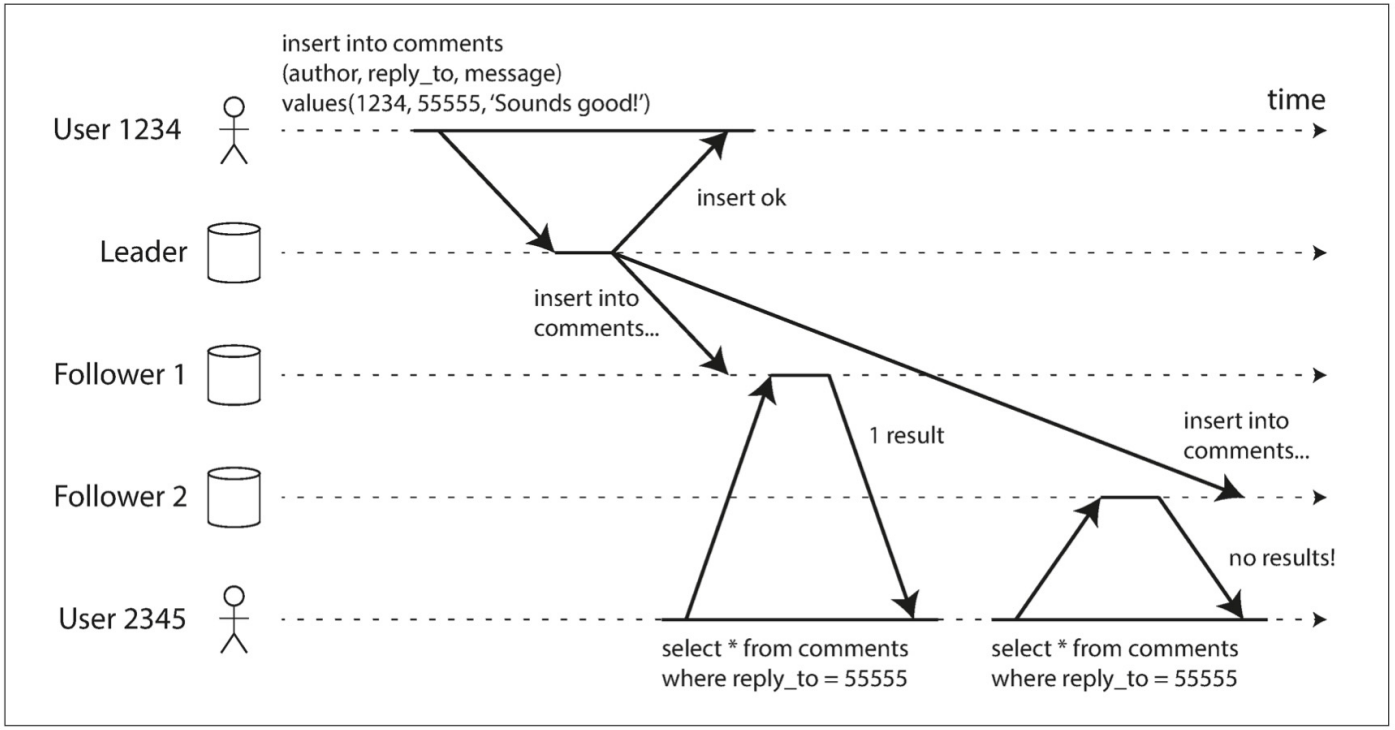
- Într-un sistem cu replicare asincronă, putem avea următorul scenariu:
  - un client trimite către lider o cerere de actualizare a unor date
  - liderul actualizează datele local, trimite confirmare către client, apoi trimite cereri de actualizare a datelor către urmăritori
  - doi clienți fac câte o cerere de citire a datelor
    - un client citește de la lider și vede datele actualizate
    - un client citește de la un urmăritor care încă nu a înregistrat actualizarea precedentă, deci vede o variantă veche a datelor
  - în cele din urmă, toți urmăritorii vor ajunge totuși să aibă aceeași versiune a datelor
- **Eventual consistency**

Probleme cu întârzierea replicării (2)



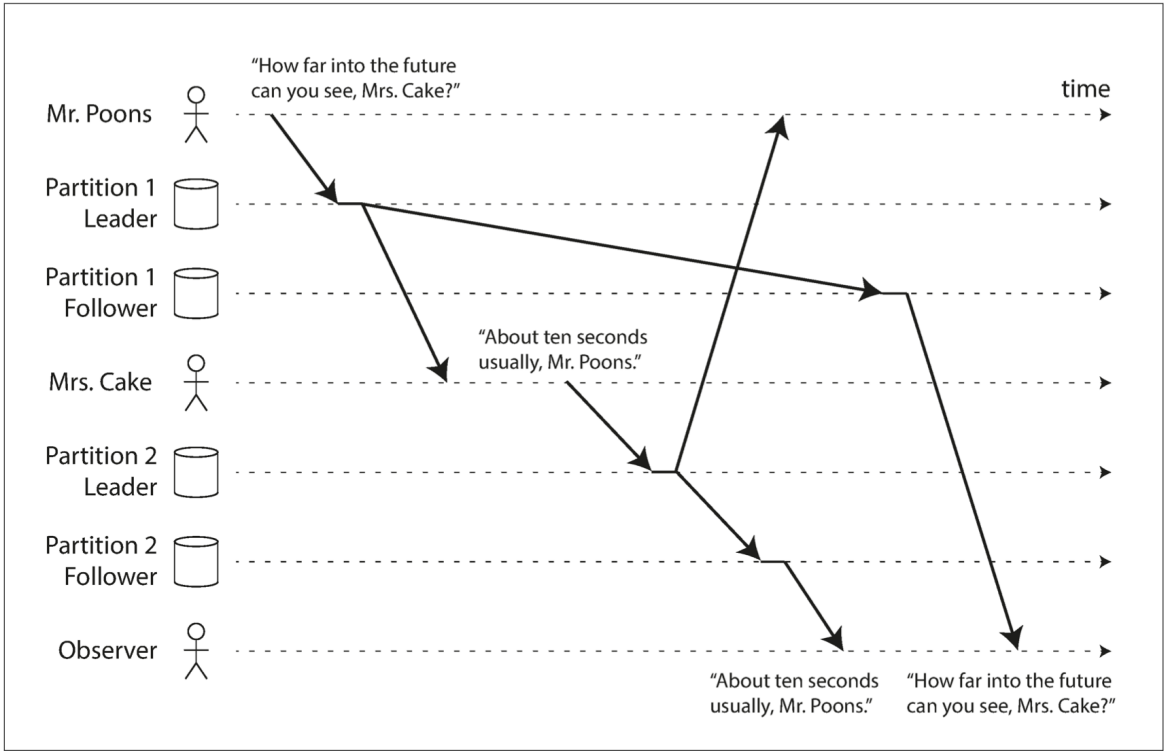
- Avem nevoie de consistență de tipul **„read-your-writes”**
  - cum am putea să o implementăm?
    - când citim ceva modificat de user, citim direct de la lider
    - pentru X secunde de la ultima actualizare a datelor, citim direct de la lider
    - se stochează un timestamp al ultimei actualizări a datelor, și se citește de la replici care au date mai recente decât acel timestamp

Probleme cu întârzierea replicării (3)



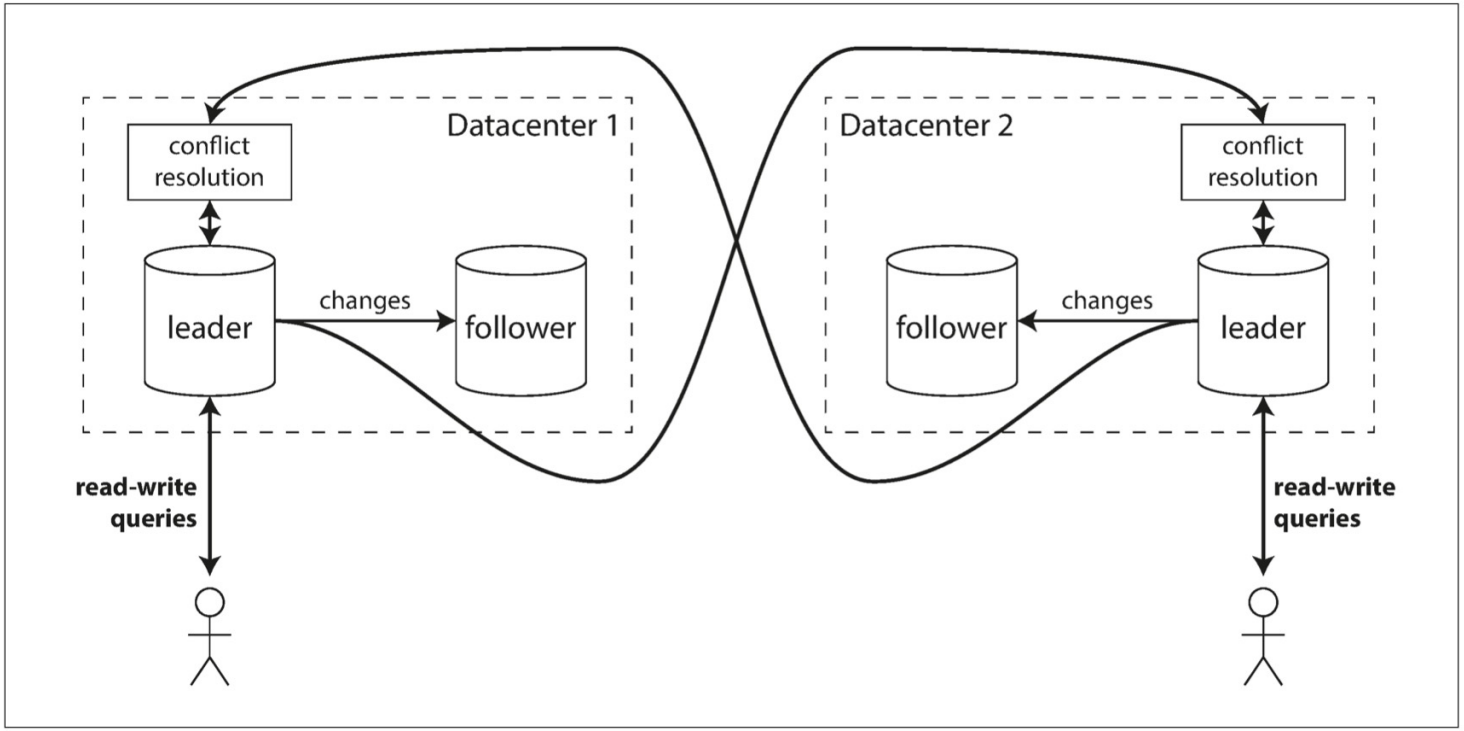
- Avem nevoie de o garanție de tipul **„monotonic reads”**
  - cum am putea să o implementăm?
    - un user face întotdeauna citiri de la aceeași replică

Probleme cu întârzierea replicării (4)



- Avem nevoie de garanție de tipul **„consistent prefix reads”**
  → dacă o secvență de scrieri are loc într-o anumită ordine, oricine citește datele trebuie să le vadă în aceeași ordine
  - cum am putea să o implementăm?
    - orice scrieri legate cauzal trebuie făcute pe aceeași partiție/replică

Replicare cu mai mulți lideri

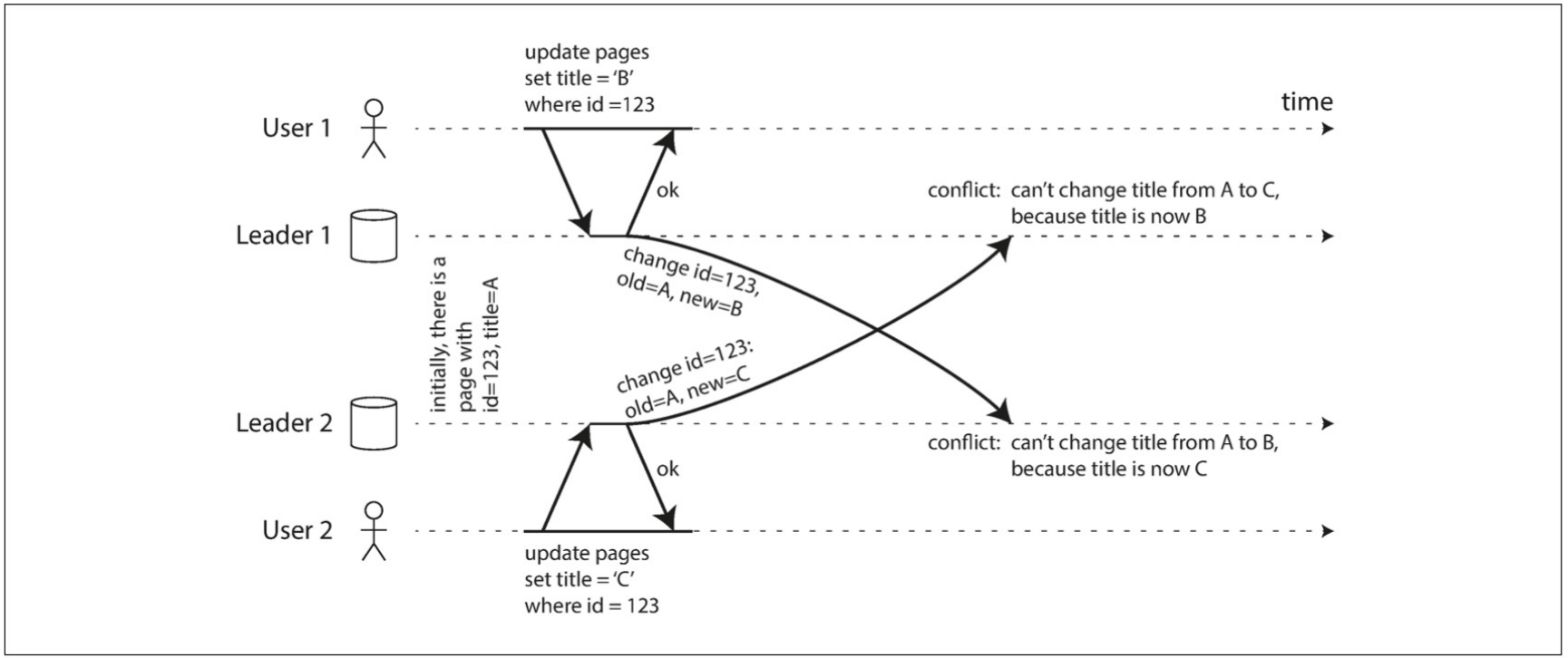


- Utilizată de obicei atunci când avem mai multe centre de date
- Mai multe replici sunt **lider** → toate cererile de scriere se fac către un lider, care informează urmăritorii săi și ceilalți lideri de modificări
- Toate celelalte replici sunt **urmăritori** ai câte unui lider → un lider informează toți urmăritorii săi despre fiecare scriere
- Un client care vrea să citească date poate trimite cererea către orice replică

Replicare cu mai mulți lideri (2)

- Avantaje:
  - Performanță → fiecare scriere este procesată în cel mai potrivit centru de date
  - Toleranța la defecte la nivel de centru de date
  - Toleranța la probleme de rețea
- Dezavantaje?
  - Aceleași date pot fi modificate concurent în mai multe centre de date diferite
- Cazuri de utilizare:
  - sisteme cu mai multe centre de date
  - clienți cu operare offline
  - editare colaborativă

Tratarea conflictelor de scriere



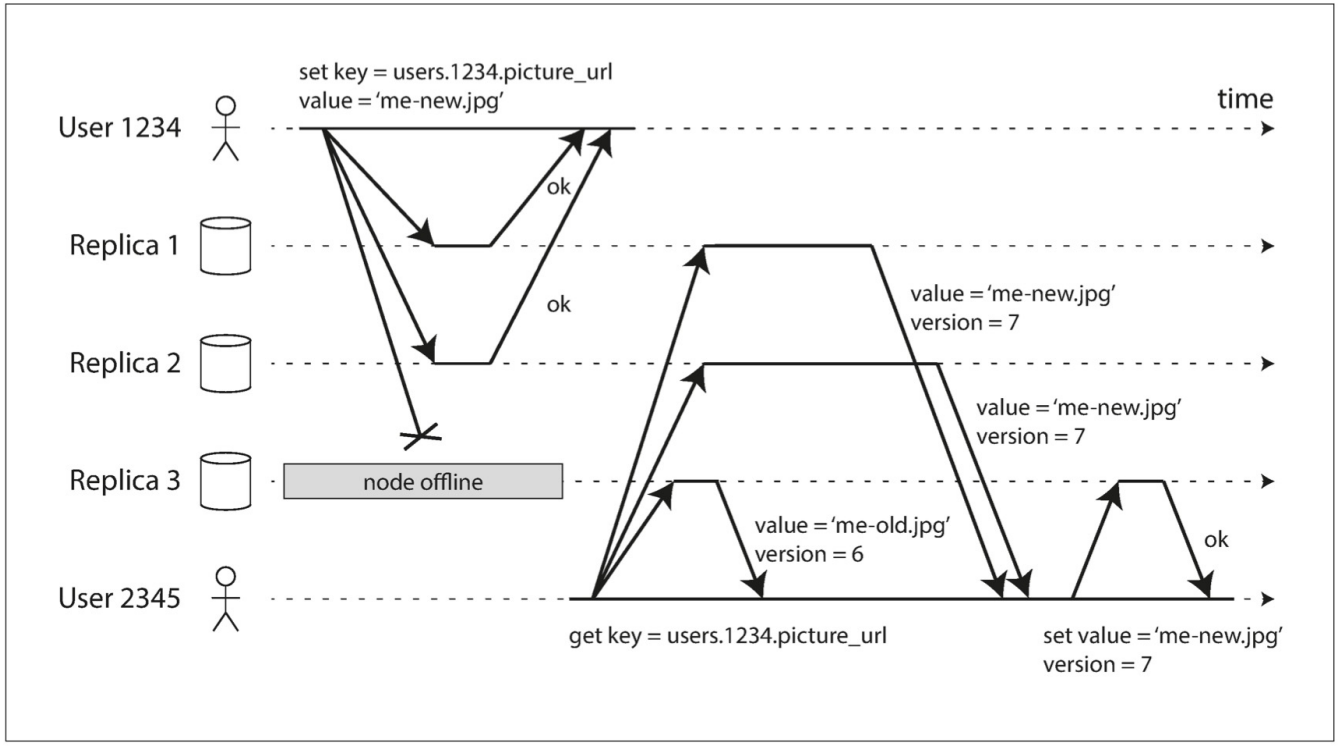
- Doi utilizatori modifică titlul unei pagini simultan
- Fiecare trimite cererea de modificare către un lider diferit
- Scrierile sunt înregistrate și confirmate de către lideri
- Când liderii încercă să înregistreze modificările unul la celălalt, apare un conflict

Tratarea conflictelor de scriere (2)

- Potențiale soluții:
  - Realizarea sincronă a detecției conflictelor → ineficientă
  - Evitarea conflictelor → toate scrierile pentru o înregistrare se realizează prin același lider
  - Rezolvarea automată a conflictelor
    - se dorește să se ajungă la o stare **convergentă**
    - moduri de realizare:
      - fiecare scriere are câte un ID unic, se păstrează doar scrierea cu ID-ul maxim
      - fiecare replică are câte un ID unic, se prioritizează scrierile de la o replică mai mare
      - se agregă valorile scrise
      - se înregistrează conflictul și se execută apoi cod pentru a-l rezolva

Replicare fără lider

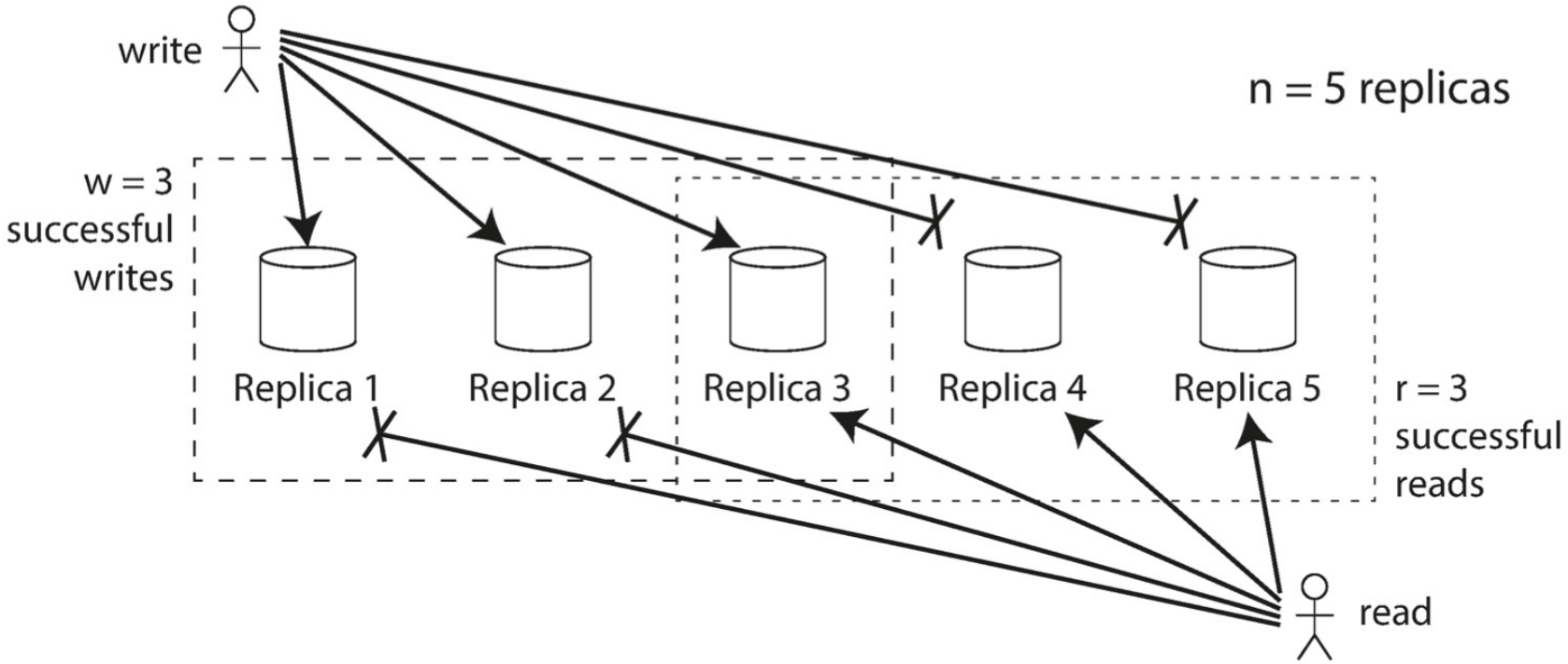
- Detalii
  - orice replică poate accepta scrieri de la orice client
  - în general, un client trimite o cerere de scriere către mai multe replici
  - **cererile de citire sunt și ele trimise către mai multe replici**



Cvorum la citire și scriere

- Datele problemei
  - *n* replici
  - fiecare scriere trebuie confirmată de *w* replici
  - fiecare citire trebuie confirmată de *r* replici
  - cererile de scriere și citire se fac către toate *n* nodurile în paralel
- Ce dorim?
  - să avem certitudinea că primim cea mai recentă versiune a datelor
- Care este condiția?
  - $w + r > n$
  - de obicei, *n* este un număr impar (3 sau 5), iar $w = r = (n + 1) / 2$
  - această condiție permite toleranța la noduri nedisponibile

Cvorum la citire și scriere (2)



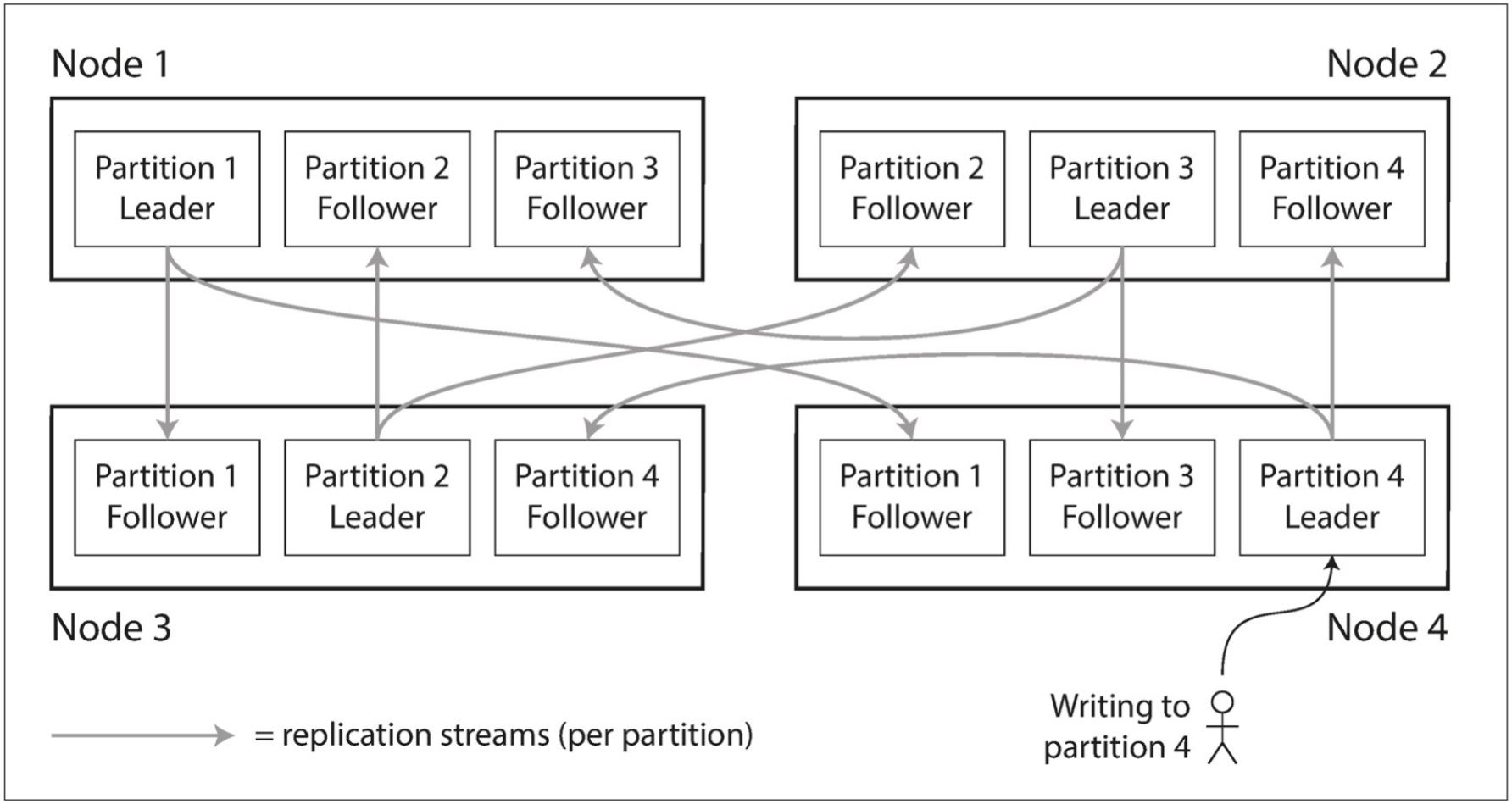
Comparație

- Replicare cu un lider
  - un singur lider care primește toate cererile de scriere
  - ușor de înțeles, nu necesită rezolvarea de conflicte
  - pot apărea probleme atunci când noduri devin indisponibile
- Replicare cu mai mulți lideri
  - mai mulți lideri, oricare din ei poate primi cererile de scriere
  - necesită rezolvare de conflicte, oferă garanții slabe de consistență
  - mai robustă în fața erorilor și a altor probleme
- Replicare fără lider
  - bazată pe cvorum
  - oferă garanții slabe de consistență
  - necesită detecție și rezolvare de conflicte

Partiționare

- *Sharding*
- Împărțirea datelor de dimensiuni mari în mai multe partiții
- Fiecare unitate de date va aparține unei singure partiții
- Motivul principal este **scalabilitatea**
- Partiționarea este combinată de obicei cu replicarea → copii ale fiecărei partiții sunt stocate pe mai multe noduri
- Un nod poate stoca mai mult de o partiție

Partiționare (2)



Partiționarea datelor de tip cheie-valoare

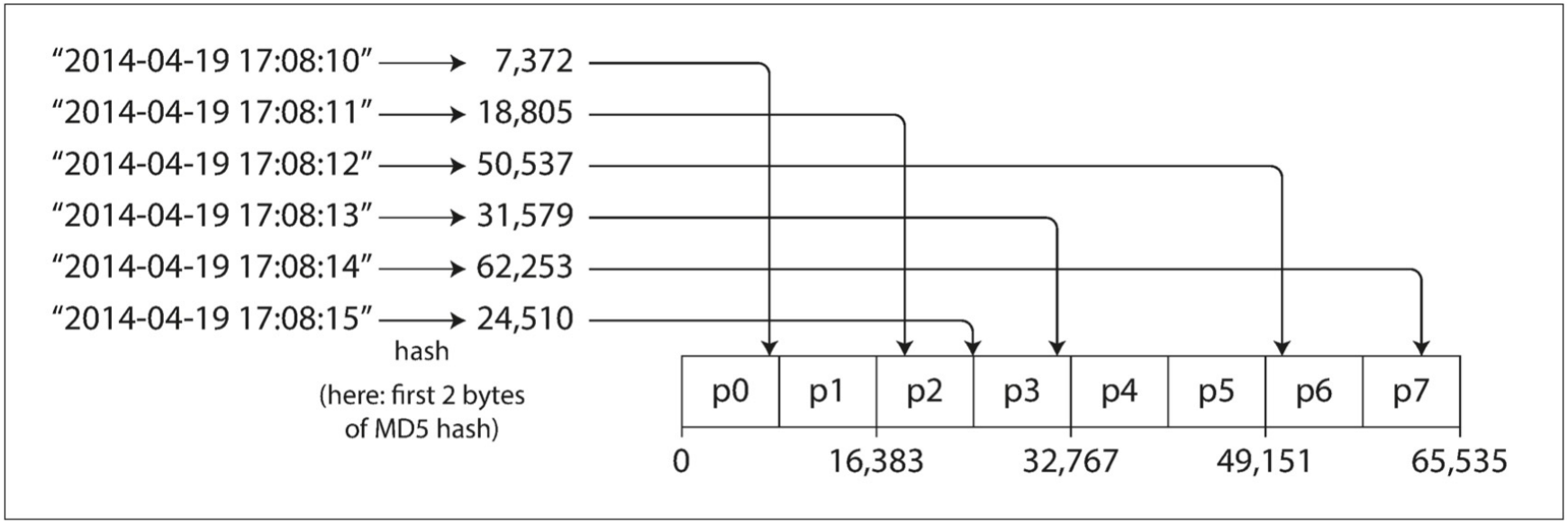
- Pentru eficiență, datele ar trebui împărțite cât mai echitabil pe noduri
- Dacă partiționarea nu este echitabilă, o numim **asimetrică** *(skewed)*
- O partiție cu o încărcare mult mai mare decât a celorlalte partiții se numește *hot spot*
- Dacă dorim o partiționare uniformă, putem asigna date către noduri în mod aleator
- **Cum știm pe ce nod regăsim apoi datele?**
- Când avem date de tip cheie-valoare, putem folosi cheile pentru a realiza logica de partiționare

Partiționarea după intervale de chei

- Putem asigna un interval continuu de chei unui nod
- Dacă știm capetele intervalelor, putem determina ușor ce partiție conține cheia căutată
- Capetele intervalului unei partiții trebuie adaptate la date, pentru a se evita partiționarea asimetrică
- În interiorul fiecărei partiții, cheile pot fi sortate → se pot trata ușor cererile pentru anumite intervale de chei
- Cheia folosită trebuie aleasă cu grijă, pentru a se evita apariția *hot spots*

Partiționarea după hash-urile cheilor

- Pentru a se evita partiționarea asimetrică, se poate folosi o funcție de hash pentru a determina partiția corespunzătoare datelor cu o anumită cheie → fiecare partiție va fi responsabilă de o gamă de chei
- Funcția de hash nu trebuie neapărat să fie puternică d.p.d.v. criptografic → important este să **distribuie uniform** datele



Partiționarea datelor cu chei secundare

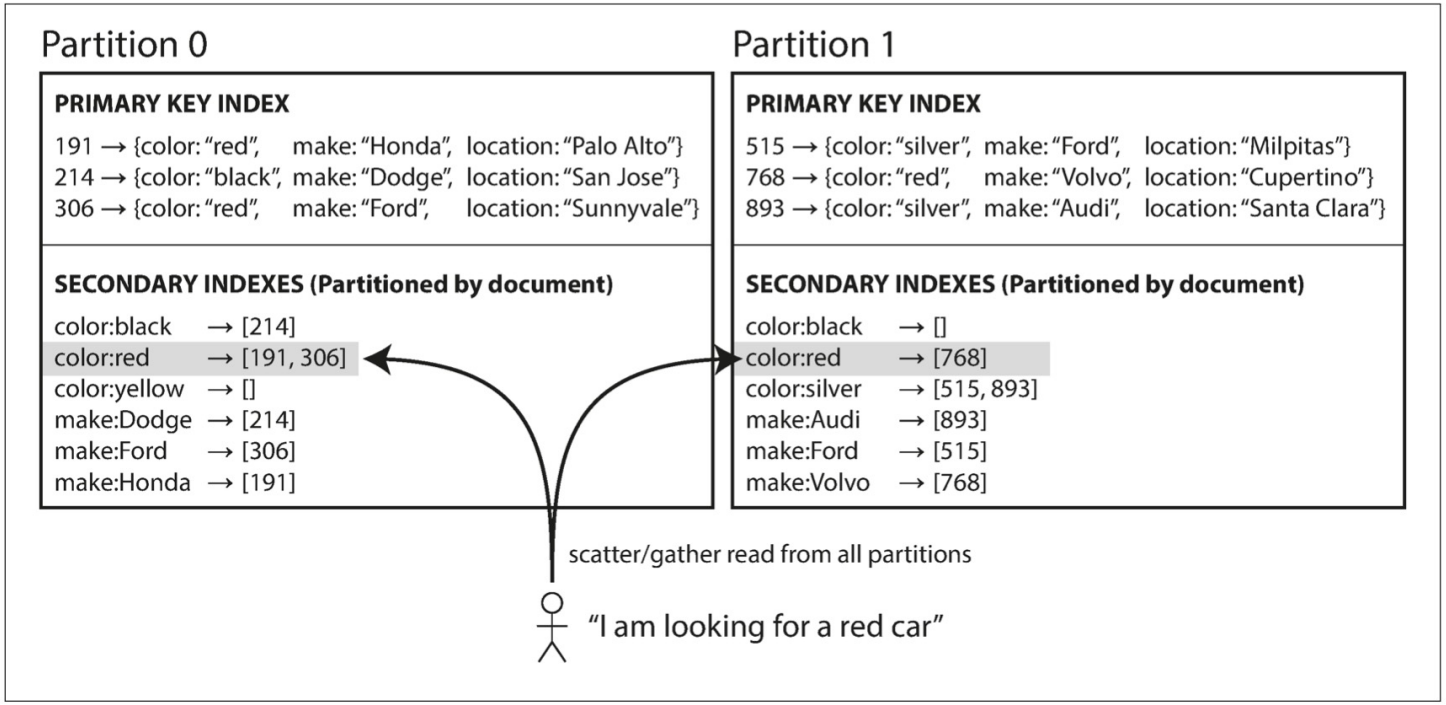
- **Cheie secundară** → nu identifică o înregistrare de date în mod unic, dar ajută la căutarea aparițiilor unei anumite valori (ex., toate mașinile de culoare roșie)
- O cheie secundară este un element foarte important în bazele de date relaționale și de documente

ID mașină	Marcă	Model	Culoare
1	Ford	Focus	Roșie
2	BMW	X3	Roșie
3	Ford	Mondeo	Albă

Cheie principală

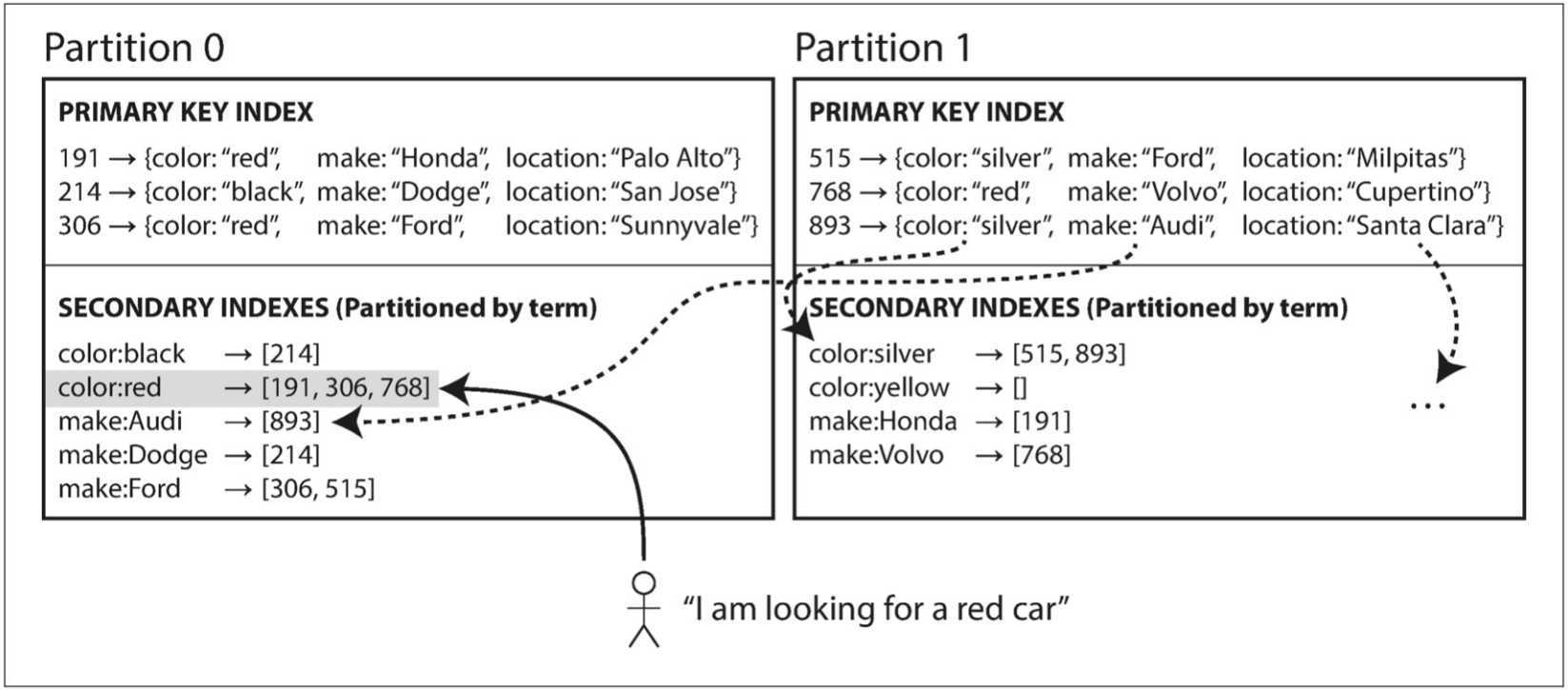
Chei secundare

Partiționarea cheilor secundare după document



- Fiecare partiție reține propriile chei secundare
- Scrierile de date se fac către o singură partiție
- Citirile de date se fac către toate partițiile

Partiționarea cheilor secundare după termen

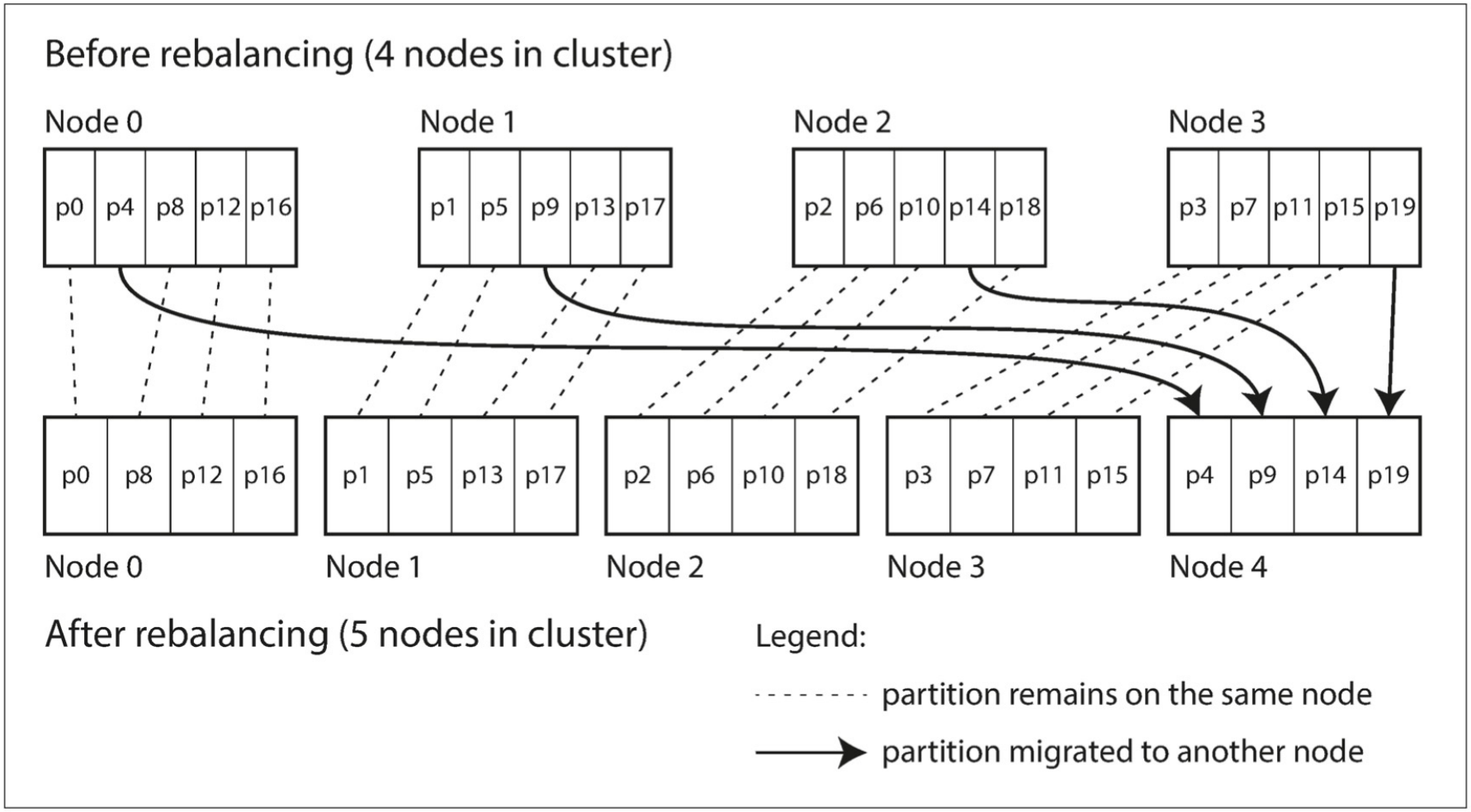


- Se creează un index global care acoperă datele din toate partițiile, și care este la rândul lui partiționat (separat)
- Scrierile de date pot ajunge să se facă pe mai multe partiții
- Cererile de date se fac către o singură partiție

Rebalansarea partițiilor

- Necesară atunci când:
  - se adaugă sau elimină noduri noi
  - cantitatea de date crește
  - apare asimetrie la partiții
- Strategii de rebalansare:
  - partiționare cu un număr fix de partiții
  - partiționare dinamică
  - partiționare proporțională cu numărul de noduri

Partiționare cu un număr fix de partiții

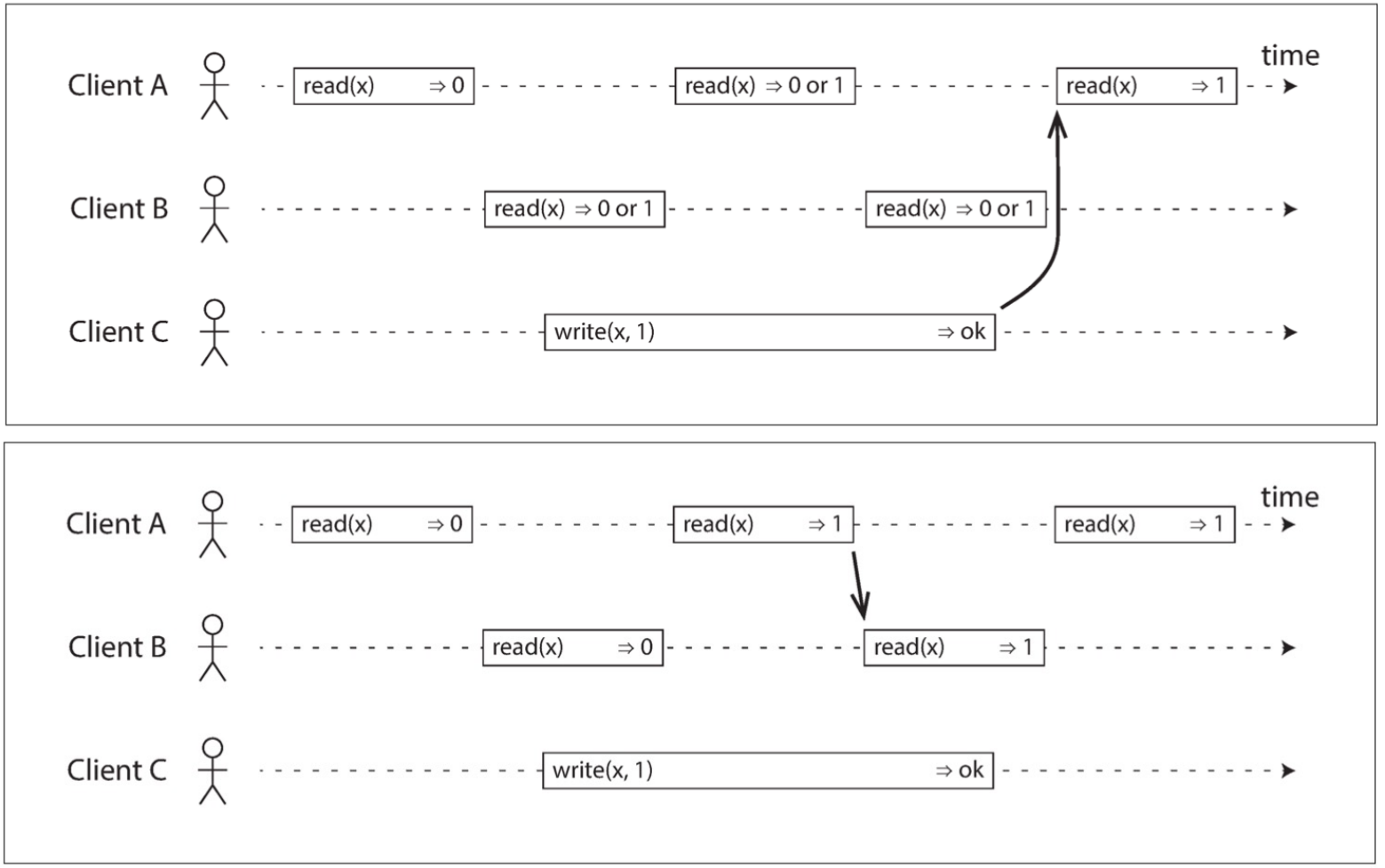


Consistența

- Toate nodurile ar trebui să vadă aceleași date în același timp (**strong/immediate/external/atomic consistency**)
- Imediat ce un client a terminat o scriere, toate citirile care urmează trebuie să vadă datele proaspăt scrise
- Majoritatea bazelor de date replicate oferă cel puțin **eventual consistency**
- Se preferă modele mai puternice de consistență, dar acestea pot duce la performanțe mai slabe sau toleranță mai redusă la defecte

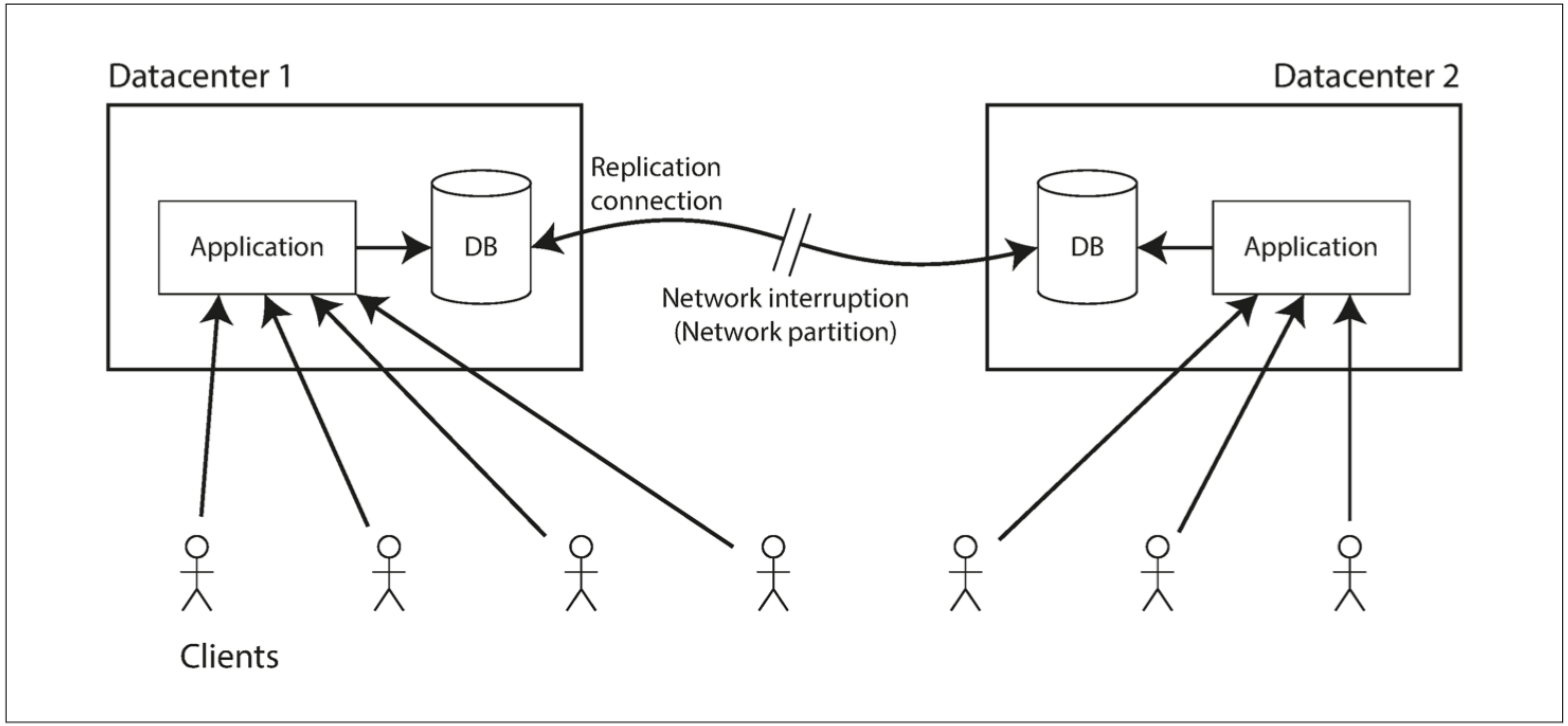
Strong consistency

- Oferă proprietatea de **liniarizabilitate**



Teorema CAP

- Propusă de Prof. Eric Brewer, descrie trade-off-urile SD
- Este imposibil pentru un sistem să furnizeze trei garanții în același timp: **C**onsistency, **A**vailability, **P**artition tolerance („alege două din trei”)



Ce spun alții...

- Coda Hale, Yammer software engineer:
  - “Of the CAP theorem’s Consistency, Availability, and Partition Tolerance, **Partition Tolerance is mandatory in distributed systems**. You cannot not choose it.”



http://codahale.com/you-cant-sacrifice-partition-tolerance/

Ce spun alții...

- Werner Vogels, Amazon CTO
  - "An important observation is that in larger distributed-scale systems, network partitions are a given; therefore, **consistency and availability cannot be achieved at the same time**."



http://www.allthingsdistributed.com/2008/12/eventually_consistent.html

Teorema CAP, 12 ani mai târziu

- Prof. Eric Brewer: părintele teoremei CAP
  - “Formularea “2 din 3” este greșită deoarece simplifică mult tensiunile dintre proprietăți...
  - **CAP interzice doar o mică parte din proiectarea distribuită**: disponibilitatea și consistența perfecte în prezența partițiilor, care sunt oricum rare.”



http://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed

Tipuri de consistență

- Strong consistency
  - După ce actualizările au loc, **orice acces ulterior** va returna **aceeași** valoare actualizată
- Weak consistency
  - **Nu există garanții** că accesări succesive vor returna valoarea actualizată
- **Eventual consistency**
  - O formă specifică a weak consistency
  - Oferă garanții că dacă nu se fac **noi actualizări** asupra obiectelor **la un moment dat** (*eventually*) toate replicile vor returna valoarea actualizată (e.g., propagarea update-urilor către replici într-o manieră lazy)

Variații ale eventual consistency

- Causal consistency
  - Procesele aflate sub o relație cauzală vor vedea date consistente
- Read-your-write consistency
  - Un process ce va accesa datele anterior scrise de el nu va vedea vreodată o valoare neactualizată
- Session consistency
  - Pe durata unei sesiuni, sistemul garantează o consistență read-your-write
  - Garanții oferite între sesiuni ce nu se suprapun

Variații ale eventual consistency (2)

- Monotonic read consistency
  - Dacă un process a văzut o valoare particulară a unei date, orice procese subsecvente nu vor returna valori mai vechi
- Monotonic write consistency
  - Sistemul garantează serializarea scrierilor de către *același* proces
- În realitate
  - Orice combinație de aceste proprietăți poate fi garantată
  - Monotonic reads și read-your-writes sunt cele mai dorite pentru sistemele distribuite

Consens

- Informal, consensul se referă la acțiunea de a face mai multe noduri să fie de acord asupra unei decizii
- Formal, consensul se definește astfel: **unul sau mai multe noduri propun niște valori, iar algoritmul de consens decide asupra uneia din acele valori**
- Proprietățile unui algoritm de consens:
  - **acord uniform** → toate nodurile decid același lucru
  - **integritate** → niciun nod nu decide de două sau mai multe ori
  - **validitate** → dacă un nod decide asupra unei valori *v*, acea valoare a fost propusă de un nod
  - **terminare** → fiecare nod decide în cele din urmă asupra unei valori

Consens (2)

- Algoritmi de consens toleranți la defecte: Viewstamped Replication (VSR), Paxos, Raft, Zab, etc.
- Detalii algoritmi de consens:
  - se definesc **epoci** → fiecare epocă nouă are un ID incremental; într-o epocă, liderul este unic
  - când liderul curent este considerat nefuncțional, se trece într-o epocă nouă și se pornește un vot între noduri pentru a alege noul lider
  - dacă este un conflict între doi lideri din două epoci diferite, liderul cu un ID de epocă mai mare câștigă
  - pentru orice decizie pe care un lider vrea sa o facă, trebuie să trimită valoarea propusă celorlalte noduri și să aștepte ca un cvorum de noduri să răspundă afirmativ
  - există deci **două runde de vot**: 1) pentru a alege liderul, 2) pentru a vota asupra propunerii unui lider

Limitările algoritmilor de consens

- Procesul prin care nodurile votează propuneri este similar cu replicarea sincronă
- Necesită întotdeauna o majoritate strictă
- Majoritatea algoritmilor de consens presupun că există o mulțime fixată de noduri care participă la votare, ceea ce înseamnă că nu este ușor să adăugăm sau eliminăm noduri din cluster
- Se bazează în general pe timeout-uri pentru a detecta nodurile picate

Sumar

- Replicare
- Partiționare
- Consistență
- Consens